

## Race Condition and Mutex Synchronization

### Why Does a Race Condition Occur?

A race condition occurs when multiple threads access and modify a shared resource simultaneously without proper synchronization.

In this program, all threads increment the same global counter variable. The increment operation:

```
counter++;
```

is not executed as a single atomic operation. Internally, it involves three steps:

1. Read the current value of the counter.
2. Increment the value.
3. Write the updated value back to memory.

When multiple threads perform these steps at the same time, their operations may overlap. For example:

- Thread A reads counter = 100
- Thread B reads counter = 100
- Thread A increments and writes 101
- Thread B increments and writes 101

Although two increments were performed, the counter increases by only one. As a result, some updates are lost, and the final counter value becomes incorrect and unpredictable.

This situation is called a **race condition** because the final result depends on the timing and execution order of the threads.

---

### How Does a Mutex Solve the Problem?

A mutex (Mutual Exclusion Lock) is a synchronization mechanism that ensures only one thread can access a critical section at a time.

In the synchronized version of the program, the counter update is protected using:

```
pthread_mutex_lock(&mutex);
```

```
/* Critical Section */  
counter++;
```

```
pthread_mutex_unlock(&mutex);
```

The process works as follows:

1. A thread must acquire the mutex before entering the critical section.
2. If another thread already holds the mutex, the current thread waits.
3. Once the first thread finishes updating the counter, it releases the mutex.
4. Another waiting thread can then acquire the mutex and continue.

Because only one thread can modify the counter at any given time, no updates are lost and the final counter value is always correct.

---

## Comparison

### Without Mutex

Multiple threads access the counter simultaneously

Race condition occurs

Updates may be lost

Incorrect final counter value

Unpredictable behavior

### With Mutex

Only one thread accesses the counter at a time

Race condition is prevented

All updates are preserved

Correct final counter value

Predictable and reliable behavior

---

## Conclusion

The unsynchronized version of the program demonstrates how concurrent access to shared data can produce incorrect results due to race conditions. By using a mutex to protect the critical section, thread execution is properly synchronized, ensuring data consistency and the correct final value of the shared counter.